

DDL Command – CREATE & Schema Definition

1. THE CREATE COMMAND OVERVIEW

CREATE is a foundational *Data Definition Language (DDL)* command used to build database objects inside an RDBMS. It defines the physical and logical structure (schema) without populating actual data rows.

Database Objects Created via CREATE

- **TABLE:** Primary structured storage container
- **DATABASE / SCHEMA:** Higher-level logical namespace
- **VIEW:** Virtual query-based table
- **SYNONYM:** Alias for database objects

Advanced Objects

- **TRIGGER:** Event-driven automated procedures
- **PROCEDURE & FUNCTION:** Executable code blocks
- **INDEX & CURSOR:** Performance and traversal structures

2. CREATE TABLE SYNTAX & STRUCTURE

Before storing relational data, the schema layout must be explicitly defined using column names and data types.

Sample Data Target:

ID	Name	Address
1	Ram	Delhi
2	Varun	Mumbai
3	Ravi	Koppal
4	Amrit	Bengaluru

Generic Syntax

```
CREATE TABLE table_name (  
    column1 data_type,  
    column2 data_type,  
    ...  
);
```

Concrete Implementation

```
CREATE TABLE Employee (  
    ID INT,  
    Name VARCHAR(20),  
    Address VARCHAR(30)  
);
```

3. NAMING RULES FOR DATABASE OBJECTS

Strict naming conventions prevent syntax parsing errors during query execution:

-  **Avoid Spaces:** `CREATE TABLE Employee Details;` (Generates syntax error without escaping)
-  **Single Word:** `CREATE TABLE Employee;`
-  **Underscore Separator:** `CREATE TABLE Employee_Details;`

4. COLUMN DEFINITIONS & DATA TYPES ★

Every column definition requires two mandatory components: **Column Identifier + Data Type Specification**.

Column Name	Data Type	Domain / Permitted Data
ID	INT / NUMBER	Numeric integer values without decimals
Name	VARCHAR(20)	Variable-length character strings (Up to 20 chars)
Address	VARCHAR(30)	Variable-length character strings (Up to 30 chars)

5. SEMICOLONS & SYNTAX RULES ⚠

6. DDL (CREATE) VS. DML (INSERT) ★★★

Distinguishing structural declaration from record manipulation is one of the most tested concepts in exams.

Parameter	CREATE (DDL)	INSERT (DML)
Scope	Data Definition Language	Data Manipulation Language
Target	Allocates structural schema & metadata	Populates actual data rows into structure
Operation	CREATE TABLE Employee (ID INT...);	INSERT INTO Employee VALUES (1, 'Ram'...);
Analogy	Building the empty house frame	Moving furniture into the house

7. INSPECTING SCHEMA STRUCTURE: DESCRIBE COMMAND

Once a table is created, developers use inspection commands to verify column structures, data types, and default rules.

Execution Statement

```
DESCRIBE Employee;
-- OR
DESC Employee;
```

Sample Output View

Name	Null?	Type
ID	NOT NULL	NUMBER
NAME		VARCHAR2(20)
ADDRESS		VARCHAR2(30)

8. COLUMN CONSTRAINTS & ORACLE DATA TYPES

Constraints restrict permitted values to enforce data validity during creation:

- **NOT NULL Constraint:** Ensures column cannot hold unassigned/empty NULL values.
- **DEFAULT Constraint:** Supplies fallback values when no value is provided during insertion.

```
CREATE TABLE Employee (
    ID INT NOT NULL PRIMARY KEY,
    Name VARCHAR2(20) NOT NULL,
    Address VARCHAR2(30) DEFAULT 'Unknown'
);
```

Oracle Dialect Details & Precision Mechanics

- **VARCHAR2 vs VARCHAR:** Oracle specifically uses VARCHAR2 for variable-length character strings.
- **Precision and Scale:** Denoted as NUMBER(p, s). For example, NUMBER(10, 2):
 - **Precision (10):** Total number of digits (before and after decimal).
 - **Scale (2):** Total number of digits allowed to the right of the decimal point (e.g., 12345678.90).

9. COMPLETE EXECUTION PIPELINE & SUMMARY MAP

```
[ Step 1: DDL Execution ] CREATE TABLE Employee (ID INT, Name VARCHAR2(20), Address VARCHAR2(30)); | ▼ (Outputs: "Table Created") [ Step 2: Schema Verification ] DESCRIBE Employee; | ▼ (Inspects Columns, Types, & Nullability) [ Step 3: DML Data Population ] INSERT INTO Employee VALUES (1, 'Ram', 'Delhi'); | ▼ (Outputs: "1 row created")
```



Final Exam Cheat-Sheet Summary

CREATE = Defines Schema | **DESCRIBE** = Displays Schema | **INSERT** = Populates Data